
CLYXN × SORXN

ELECTRON.JS WORKSHOP

Project Setup & Core Files

A practical workshop guide for creating an Electron.js project and understanding the core files required to configure and run a desktop application.

MAIN.JS PRELOAD.JS PACKAGE.JSON

CLYXN × SORXN

Workshop Documentation

ELECTRON.JS

Project Setup & Core Files

SECTION 01

Workshop GitHub Repository

Accessing the public workshop repository and using the essential Git commands required to clone, update, and open the project.

1. Workshop GitHub Repository

All source code, project files, and workshop updates will be maintained in the official public GitHub repository for the Electron.js Workshop.

Students will use this repository throughout the workshop to access the latest version of the project.

1.1 Public Repository

The workshop repository is publicly available at:

<https://github.com/KshitijSawant1/ElectronJS-Workshop-NirmalaCollege-2026>

Repository:

```
KshitijSawant1/ElectronJS-Workshop-NirmalaCollege-2026
```

The repository can be accessed without requesting repository access.

1.2 Clone the Repository

The repository needs to be cloned only once when setting up the workshop project for the first time.

Open the Visual Studio Code terminal, Command Prompt, PowerShell, or macOS Terminal and navigate to the location where the workshop project should be stored.

Run:

```
git clone https://github.com/KshitijSawant1/ElectronJS-Workshop-NirmalaCollege-2026.git
```

Git will download the repository and create the following project folder:

```
ElectronJS-Workshop-NirmalaCollege-2026/
```

1.3 Open the Project Directory

Move into the cloned repository:

```
cd ElectronJS-Workshop-NirmalaCollege-2026
```

To open the project directly in Visual Studio Code, run:

```
code .
```

1.4 Check Repository Status

The current Git repository status can be checked using:

```
git status
```

This command displays the current branch and any local changes made to the workshop files.

1.5 Pull the Latest Workshop Updates

The workshop repository may be updated as the sessions progress.

Before starting a new session, open the existing project directory and run:

```
git pull origin main
```

This downloads and integrates the latest changes from the repository's **main** branch into the local project.

1.6 Workshop Git Workflow

After the repository has been cloned, the standard workflow before each workshop session is:

```
cd ElectronJS-Workshop-NirmalaCollege-2026  
git pull origin main  
code .
```

The **git clone** command should not be repeated for every session.

Use **git pull origin main** to update the existing local copy of the repository.

SECTION 02

Creating the Grovio Project

Creating the Next.js application, installing Electron development packages, and preparing the initial Grovio project structure.

2. Creating the Grovio Project

Grovio will initially be created as a Next.js application. Next.js provides the React-based application structure, routing system, styling environment, and development server used to build the Grovio interface.

Electron.js will then be integrated with this Next.js application to convert it into a desktop application.

2.1 Check Node.js and npm

Before creating the project, confirm that Node.js and npm are available on the system.

Open Command Prompt, PowerShell, the Visual Studio Code terminal, or macOS Terminal and run:

```
node -v
npm -v
```

Both commands should return valid version numbers.

For the current Next.js development environment, Node.js version **20.9 or later** should be available.

If both commands execute successfully, continue with the project setup.

2.2 Create the Next.js Project

Navigate to the directory where the workshop project should be created.

Run:

```
npx create-next-app@latest grovio
```

The Next.js setup utility will ask a series of configuration questions.

Select the following options:

Configuration	Selection
Would you like to use TypeScript?	No
Which linter would you like to use?	ESLint
Would you like to use React Compiler?	No
Would you like to use Tailwind CSS?	Yes
Would you like your code inside <code>src/</code> ?	No
Would you like to use App Router?	Yes
Customize import alias?	No

After the configuration is complete, Next.js will create the `grovio` directory and automatically install the standard dependencies required by the project.

2.3 Enter the Grovio Project

Move into the newly created project directory:

```
cd grovio
```

All subsequent npm and Electron commands should be executed from inside this directory.

2.4 Open the Project in Visual Studio Code

Open the Grovio project using:

```
code .
```

Visual Studio Code will open the complete Next.js project structure.

At this stage, the project will contain folders and files similar to:

```
grovio/  
|  
|-- app/  
| |-- globals.css  
| |-- layout.js  
| |-- page.js  
|  
|-- public/  
|-- package.json  
|-- package-lock.json  
|-- next.config.mjs  
|-- node_modules/
```

2.5 Run the Next.js Application

Before integrating Electron, verify that the Next.js application runs correctly.

Start the development server:

```
npm run dev
```

The terminal will display the local development address, normally:

```
http://localhost:3000
```

Open the address in a web browser.

If the Next.js application appears successfully, the Grovio web application environment is working correctly.

To stop the development server when required, press:

```
Ctrl + C
```

2.6 Install the Required Electron Libraries

Grovio now requires additional npm packages for desktop application development.

Install **Electron** and **Concurrently** as development dependencies:

```
npm install --save-dev electron concurrently
```

The command installs two packages:

Package	Purpose
Electron	Provides the desktop application runtime and BrowserWindow environment.
Concurrently	Allows the Next.js development server and Electron application to run from a combined development workflow.

These packages will automatically be added to the `devDependencies` section of `package.json`.

2.7 Verify the Installed Packages

Verify Electron using:

```
npx electron --version
```

A valid Electron version number should be displayed.

The installed development dependencies can also be checked using:

```
npm list electron concurrently
```

If both packages are listed, the required npm libraries have been installed successfully.

2.8 Create the Electron Directory

To keep the desktop-specific files separate from the Next.js interface, create a dedicated directory named:

```
electron
```

Inside this directory, create:

```
main.js  
preload.js
```

The project will now follow this structure:

```
grovio/  
|  
|-- app/  
| |-- page.js  
| |-- layout.js
```



```
| |-- globals.css
|
|-- components/
|
|-- public/
|
|-- electron/
| |-- main.js
| |-- preload.js
|
|-- package.json
|-- package-lock.json
|-- node_modules/
```

The purpose of the two Electron files is:

- **main.js** — Starts the Electron application and creates the desktop window.
- **preload.js** — Provides a controlled bridge between Electron and the Next.js interface.

2.9 Setup Command Reference

The following commands are used during the Grovio project setup.

2.9.1 Check Node.js Version

```
node -v
```

Displays the currently installed Node.js version.

2.9.2 Check npm Version

```
npm -v
```

Displays the currently installed npm version.

2.9.3 Create the Grovio Project

```
npx create-next-app@latest grovio
```

Creates a new Next.js application named **grovio**.

2.9.4 Enter the Project Directory

```
cd grovio
```

Moves the terminal into the Grovio project directory.

2.9.5 Open the Project in Visual Studio Code

```
code .
```

Opens the current project directory in Visual Studio Code.

2.9.6 Run the Next.js Development Server

```
npm run dev
```

Starts the Next.js development server.

The application will normally be available at:

```
http://localhost:3000
```

2.9.7 Install Electron and Concurrently

```
npm install --save-dev electron concurrently
```

Installs Electron and Concurrently as development dependencies.

2.9.8 Check Electron Version

```
npx electron --version
```

Displays the installed Electron version.

2.9.9 Verify Electron and Concurrently

```
npm list electron concurrently
```

Displays the installed Electron and Concurrently packages and their versions.

Current Stage: The Next.js application has been created and verified, and the Electron development packages are now installed. The `electron/main.js` and `electron/preload.js` files will be configured in the next section.

SECTION 03

Electron Main Process

Understanding and configuring the Electron main process responsible for creating the Grovio desktop application window.

3. electron/main.js

The `main.js` file is the entry point of the Electron application. It runs in Electron's main process and is responsible for creating the desktop window that will display the Grovio Next.js application.

The file is located at:

```
grovio/electron/main.js
```

3.1 Import Required Modules

Start by importing Electron and Node.js modules:

```
const { app, BrowserWindow } = require("electron");
const path = require("path");
```

The imported modules are used as follows:

- **app** manages the Electron application lifecycle.
- **BrowserWindow** creates the desktop application window.
- **path** creates a reliable path to the preload file.

3.2 Create the Electron Window

Create a function named `createWindow()`:

```
function createWindow() {
  const win = new BrowserWindow({
    width: 1200,
    height: 800,
  });
}
```

The **BrowserWindow** object creates the main Grovio desktop window.

The initial size of the application is:

- Width: **1200 px**
- Height: **800 px**

3.3 Connect the Preload Script

Inside **BrowserWindow**, configure the preload file:

```
webPreferences: {
  preload: path.join(__dirname, "preload.js"),
},
```

Since both files are inside the same `electron` directory, `__dirname` points to:

```
grovio/electron/
```

Therefore:

```
path.join(__dirname, "preload.js")
```

resolves to the correct preload file.

3.4 Load the Grovio Interface

During development, the Next.js application runs on:

```
http://localhost:3000
```

Load this address inside the Electron window:

```
win.loadURL("http://localhost:3000");
```

This allows Electron to display the Grovio Next.js interface as a desktop application.

3.5 Start the Electron Application

Electron should create the window only after the application is ready:

```
app.whenReady().then(createWindow);
```

The `whenReady()` method waits for Electron initialization to complete and then executes `createWindow()`.

3.6 Complete main.js

Copy the following code into:

```
grovio/electron/main.js
```

```
const { app, BrowserWindow } = require("electron");
const path = require("path");

function createWindow() {
  const win = new BrowserWindow({
    width: 1200,
    height: 800,
    webPreferences: {
      preload: path.join(__dirname, "preload.js"),
    },
  });

  win.loadURL("http://localhost:3000");
}

app.whenReady().then(createWindow);
```

3.7 What `main.js` Does

- Starts the Electron application.
- Creates the Grovio desktop window.
- Loads `preload.js`.
- Opens the Next.js application from `localhost:3000`.

SECTION 04

Electron Preload Script

Creating a controlled bridge between the Electron environment and the Grovio Next.js interface using the preload script.

4. electron/preload.js

The `preload.js` file runs before the Grovio interface is loaded inside the Electron window.

It provides a controlled way to expose selected Electron information to the Next.js interface.

The file is located at:

```
grovio/electron/preload.js
```

4.1 Import contextBridge

Import `contextBridge` from Electron:

```
const { contextBridge } = require("electron");
```

The `contextBridge` is used to expose selected values from Electron to the renderer without directly exposing the entire Electron environment.

4.2 Expose the Electron Object

Use:

```
contextBridge.exposeInMainWorld("electron", {  
  });
```

The name:

```
"electron"
```

creates an object that can later be accessed in the Next.js interface as:

```
window.electron
```

4.3 Expose the Electron Version

Add the running Electron version to the exposed object:

```
version: process.versions.electron,
```

The value:

```
process.versions.electron
```

returns the version of Electron currently running the application.

The Next.js interface can access this value using:

```
window.electron.version
```


4.4 Complete preload.js

Copy the following code into:

```
grovio/electron/preload.js
```

```
const { contextBridge } = require("electron");

contextBridge.exposeInMainWorld("electron", {
  version: process.versions.electron,
});
```

4.5 What preload.js Does

- Runs before the Next.js interface is loaded.
- Uses `contextBridge` to expose selected values.
- Creates the `window.electron` object.
- Makes the current Electron version available through `window.electron.version`.

Key Point: Only the values explicitly added inside `contextBridge.exposeInMainWorld()` become available to the Grovio interface.

SECTION 05

Package Configuration

Configuring the Electron entry point, development scripts, and required packages inside the Grovio package.json file.

5. Configuring package.json

The `package.json` file must be updated so that the Next.js development server and Electron can run together.

Open:

```
grovio/package.json
```

5.1 Add the Electron Entry Point

Add the following property after `"private": true`:

```
"main": "electron/main.js",
```

This tells Electron to start the application using:

```
electron/main.js
```

5.2 Update the Development Script

Find the existing `dev` script:

```
"dev": "next dev"
```

Replace it with:

```
"dev": "concurrently \"next dev\" \"wait-on http://localhost:3000 && electron .\""
```

This performs three operations:

- Starts the Next.js development server.
- Waits until `localhost:3000` is available.
- Starts the Electron application.

5.3 Install the Required Packages

Ensure the following development packages are installed:

```
npm install --save-dev electron concurrently wait-on
```

The packages serve the following purposes:

- **electron** — Runs the desktop application.
- **concurrently** — Runs Next.js and Electron together.
- **wait-on** — Waits for Next.js before launching Electron.

5.4 Complete package.json

After the changes, the file should contain:

```
{
  "name": "grovio",
  "version": "0.1.0",
  "private": true,
  "main": "electron/main.js",
  "scripts": {
    "dev": "concurrently \"next dev\" \"wait-on http://localhost:3000 && electron .\"",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@supabase/ssr": "^0.12.4",
    "@supabase/supabase-js": "^2.112.3",
    "next": "16.3.1",
    "react": "19.2.8",
    "react-dom": "19.2.8",
    "react-icons": "^5.7.0"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "concurrently": "^10.0.5",
    "electron": "^43.4.0",
    "eslint": "^9",
    "eslint-config-next": "16.3.1",
    "tailwindcss": "^4",
    "wait-on": "^9.1.0"
  }
}
```

5.5 Run Grovio

Save package.json and run:

```
npm run dev
```

This single command now starts both the **Next.js development server** and the **Electron desktop application**.

Required Changes: Add "main": "electron/main.js", update the dev script, and ensure electron, concurrently, and wait-on are installed.